Aluno 1: Ryan Dias da Silva | **RA:** 1051392611004 **Aluno 2:** Murilo Paes Sobrinho | **RA:** 1051392611041

Parte 1: Respostas das Questões de Múltipla Escolha

10 Questões

Questão 1 (Pilhas)

Alternativa A

Justificativa: Executando as operações passo a passo: entra 10, entra 20, sai 20 (resta 10), entra 30, entra 40, sai 40 (restam 10 e 30), entra 50. A pilha final contém 10, 30 e 50, totalizando 3 elementos com o 50 no topo.

Questão 2 (Pilhas)

Alternativa C

Justificativa: O valor -1 indica uma pilha vazia, permitindo que a primeira inserção, usando pré-incremento (++topo), ocorra de forma segura no índice 0, que é a primeira posição válida em arrays na linguagem C.

Questão 3 (Pilhas)

Alternativa D

Justificativa: A primeira string `((a+b)*c)` possui parênteses corretamente balanceados, retornando 1 (válida). A segunda string `((a+b)` possui um parêntese de abertura sem fechamento correspondente, retornando 0 (inválida).

Questão 4 (Pilhas)

Alternativa B

Justificativa: O comportamento da operação clássica `peek()` (ou top) é inspecionar e retornar o valor localizado no topo da pilha sem removê-lo ou alterar o estado da estrutura.

Questão 5 (Pilhas)

Alternativa E

Justificativa: O escalonamento de processos de CPU por ordem estrita de chegada de requisições de prioridade igual segue o comportamento de uma Fila (FIFO), não sendo uma aplicação de Pilha (LIFO).

Questão 6 (Filas)

Alternativa B

Justificativa: Após as três inserções, a fila contém [10, 20, 30]. O primeiro dequeue remove o 10. A inserção do 40 resulta em [20, 30, 40]. O segundo dequeue remove o 20. Os elementos restantes são [30, 40], sendo o 30 o elemento na frente.

Questão 7 (Filas)

Alternativa B

Justificativa: Filas estáticas lineares sem reaproveitamento sofrem de desperdício de memória nas posições iniciais. A solução é utilizar um Buffer Circular com o operador de módulo matemático (%).

Questão 8 (Filas)

Alternativa D

Justificativa: Para garantir que inserções e remoções operem em tempo constante O(1), a estrutura exige dois ponteiros: um na frente para remoções e outro no final (cauda) para inserções.

Questão 9 (Filas)

Alternativa B

Justificativa: O gerenciamento de um spooler de impressão de rede que atende requisições de múltiplos usuários obedece à ordem natural de chegada (FIFO), que é a aplicação ideal e clássica de uma fila.

Questão 10 (Filas)

Alternativa C

Justificativa: A pilha necessita de apenas um ponteiro para rastreamento (o topo), ao passo que a fila necessita de dois indicadores (início e fim) para gerenciar as duas extremidades ativas.

2.1) Script Errado 1: Script com Erro sobre Pilha (61Exemplo_Bug.c)

Pilha Estática (Vetor)

LISTA DE ERROS TÉCNICOS (PARA O PROFESSOR):

- **Falta de validação de subfluxo (Stack Underflow):** A função `pop()` não verifica se a pilha está vazia (`topo == -1`) antes de acessar o vetor. Ao desempilhar com a pilha vazia, tenta ler `pilha[-1]`, provocando leitura fora dos limites (*out-of-bounds array access*) e comportamento indefinido.
- **Corrupção do índice de controle (`topo`):** O operador de pós-decremento `topo--` decrementa `-1` para `-2`, corrompendo o estado interno da pilha e desregulando qualquer inserção ou remoção futura.
- **Execução de `pop()` prematuro na `main()`:** A função principal executa `pop()` imediatamente após a inicialização sem nenhuma inserção prévia (`push()`), disparando o erro de Underflow de imediato.

Listagem 1.2: Código Correto — **61Exemplo_Corrigido.c**

```c
#include <stdio.h>
#define MAX 5

int pilha[MAX];
int topo = -1;

int push(int valor) {
    if (topo >= MAX - 1) {
        printf("Erro: Pilha cheia (Stack Overflow)!\n");
        return 0;
    }
    pilha[++topo] = valor;
    return 1;
}

int pop() {
    // Validacao obrigatoria de subfluxo (Stack Underflow)
    if (topo == -1) {
        printf("Erro: Pilha vazia (Stack Underflow)!\n");
        return -1;
    }
    return pilha[topo--];
}

int main() {
    push(10);
    push(20);
    printf("Desempilhando: %d\n", pop()); // Retorna 20
    printf("Desempilhando: %d\n", pop()); // Retorna 10
    printf("Desempilhando: %d\n", pop()); // Dispara aviso seguro de Underflow
    return 0;
}
```

2.2) Cenário 2: Script com Erro sobre Pilha Dinâmica (62Exemplo_Bug.c)

Pilha Dinâmica Encadeada

LISTA DE ERROS TÉCNICOS (PARA O PROFESSOR):

1. **Vazamento de Memória (*Memory Leak*) no `pop()`:** O ponteiro `topo` é avançado para `topo->proximo` sem antes desalocar o nó desempilhado com `free()`. O bloco de memória permanece alocado no *Heap* sem qualquer referência, impossibilitando sua recuperação e consumindo memória indefinidamente.
2. **Ausência de checagem do retorno de `malloc()` no `push()`:** A função não verifica se `novo == NULL`. Caso ocorra falta de memória livre no sistema operacional, a instrução `novo->dado = valor;` tentará desreferenciar um ponteiro nulo, provocando encerramento abrupto por falha de segmentação (*Segmentation Fault*).
3. **Ambiguidade no sentinela de erro e retorno descartado:** Retornar `-1` quando a pilha está vazia é ambíguo caso `-1` seja um valor válido armazenado na pilha. Além disso, na `main()`, a chamada `pop()` tem seu retorno descartado sem exibição ou validação.

Listagem 2.2: Código Correto — **62Exemplo_Corrigido.c**

```c
#include <stdio.h>
#include <stdlib.h>

typedef struct No {
    int dado;
    struct No* proximo;
} No;

No* topo = NULL;

void push(int valor) {
    No* novo = (No*)malloc(sizeof(No));
    if (novo == NULL) {
        printf("Erro: Falha na alocacao de memoria!\n");
        return;
    }
    novo->dado = valor;
    novo->proximo = topo;
    topo = novo;
}

int pop() {
    if (topo == NULL) {
        printf("Erro: Pilha vazia (Underflow)!\n");
        return -1;
    }
    No* temp = topo;           // 1. Armazena ponteiro temporario do no
    int valor = temp->dado;    // 2. Extrai o dado do elemento
    topo = topo->proximo;      // 3. Desconecta o no e atualiza o topo
    free(temp);                // 4. Libera a memoria (previne Memory Leak)
    return valor;
}

int main() {
    push(10);
    push(20);
    printf("Desempilhado: %d\n", pop()); // 20
    printf("Desempilhado: %d\n", pop()); // 10
    return 0;
}
```

2.3) Cenário 3: Script com Erro sobre Fila Linear (66Fila_Bug.c)

Fila Estática Linear vs. Circular

LISTA DE ERROS TÉCNICOS (PARA O PROFESSOR):

4. **Estouro de Limites do Vetor (*Array Out-of-Bounds / Buffer Overflow*):** No `enqueue()`, o avanço estritamente linear `fila[++fim] = v` faz `fim` atingir o índice 3 após 3 inserções e 1 remoção. Mesmo existindo vaga livre no índice `0` (`total < MAX`), o código tenta escrever em `fila[3]`, fora dos limites do vetor.
5. **Inutilização de posições liberadas e avanço linear contínuo de `inicio`:** No `dequeue()`, o incremento `inicio++` avança indefinidamente para a direita, tornando o início do vetor inacessível e eventualmente extrapolando a capacidade física do array em remoções subsequentes.
6. **Ausência de Aritmética Modular (Buffer Circular):** A estrutura não utiliza o operador de módulo (`% MAX`) para reutilizar os índices de forma circular (`fim = (fim + 1) % MAX` e `inicio = (inicio + 1) % MAX`), requisito fundamental para filas estáticas eficientes sem deslocamento de dados.

Listagem 3.2: Código Correto — **66Fila_Corrigido.c**

```c
#include <stdio.h>
#define MAX 3

int fila[MAX];
int inicio = 0, fim = -1, total = 0;

void enqueue(int v) {
    if (total >= MAX) {
        printf("Fila Cheia!\n");
        return;
    }
    fim = (fim + 1) % MAX; // Avanco circular seguro do indice fim
    fila[fim] = v;
    total++;
}

int dequeue() {
    if (total == 0) {
        printf("Fila Vazia!\n");
        return -1;
    }
    int v = fila[inicio];
    inicio = (inicio + 1) % MAX; // Avanco circular seguro do indice inicio
    total--;
    return v;
}

int main() {
    enqueue(10); enqueue(20); enqueue(30);
    printf("Desenfileirado: %d\n", dequeue()); // Libera a vaga no indice 0
    enqueue(40);                               // Insere com sucesso no indice 0 reaproveitado!
    printf("Desenfileirado: %d\n", dequeue()); // 20
    printf("Desenfileirado: %d\n", dequeue()); // 30
    printf("Desenfileirado: %d\n", dequeue()); // 40
    return 0;
}
```

2.4) Cenário 4: Script com Erro sobre Fila Encadeada Dinâmica (67Fila_Bug.c)

Fila Dinâmica Encadeada

LISTA DE ERROS TÉCNICOS (PARA O PROFESSOR):

7. **Vazamento de Memória (*Memory Leak*):** O nó removido é desconectado de `inicio`, mas a memória alocada dinamicamente nunca é liberada com `free()`. O bloco permanece inacessível e retido no *Heap* durante toda a execução.
8. **Ponteiro Pendente e Estado Inconsistente (*Dangling Pointer*):** Quando a fila possui apenas 1 elemento (`inicio == fim`), o código executa apenas `inicio = NULL`, mantendo `fim` apontando para o nó desalocado/desconectado. Essa inconsistência quebra a integridade da estrutura (`inicio == NULL && fim != NULL`) e gera falhas críticas nas próximas inserções. Ambos devem ser anulados: `inicio = NULL; fim = NULL;`.
9. **Falta de retorno do dado desenfileirado:** A função possui retorno `void` e descarta o valor contido no nó, violando a especificação padrão de `dequeue()`, que deve entregar o elemento removido ao código chamador.

Listagem 4.3: Código Correto — **67Fila_Corrigido.c**

```c
#include <stdio.h>
#include <stdlib.h>

typedef struct No {
    int dado;
    struct No* prox;
} No;

No* inicio = NULL;
No* fim = NULL;

void enqueue(int valor) {
    No* novo = (No*)malloc(sizeof(No));
    if (novo == NULL) return;
    novo->dado = valor;
    novo->prox = NULL;
    if (fim == NULL) {
        inicio = fim = novo;
    } else {
        fim->prox = novo;
        fim = novo;
    }
}

int dequeue() {
    if (inicio == NULL) {
        printf("Erro: Fila Vazia (Underflow)!\n");
        return -1;
    }
    No* temp = inicio;          // 1. Guarda referencia temporaria do no
    int valor = temp->dado;     // 2. Salva o dado para retorno
    inicio = inicio->prox;      // 3. Avanca o inicio para o proximo

    if (inicio == NULL) {       // 4. Sincroniza fim se a fila esvaziou
        fim = NULL;
    }

    free(temp);                 // 5. Libera a memoria alocada (previne Memory Leak)
    return valor;
}

int main() {
    enqueue(10); enqueue(20);
    printf("Desenfileirado: %d\n", dequeue()); // 10
    printf("Desenfileirado: %d\n", dequeue()); // 20 (inicio e fim tornam-se NULL)
    return 0;
}
```